

Threat Model Report

Enterprise DevSecOps Security Lab - system-level threat model, trust boundaries, assets, attack surfaces, controls, and risk treatment.

Project	Enterprise DevSecOps Security Lab
Document owner	Jagriti Banerjee
Document type	Threat model / security architecture assessment
Scope	Vulnerable Flask app, CI/CD pipeline, container registry, Kubernetes runtime, GitOps, supply chain security, detection and observability
Methodology	Data-flow modeling, trust boundary analysis, STRIDE classification, control validation, residual-risk review
Output artifacts	threat_model.pdf, threat_model/stride_threat_model.pdf, diagrams/data-flow-diagram.svg

Classification: Private lab documentation / portfolio evidence. This document is designed for security review, recruiter evaluation, and technical audit walkthroughs.

Document contents

Section group	What the reviewer will find
Architecture and scope	System overview, trust boundaries, assets and data-flow diagram.
Threat analysis	STRIDE categories, attack paths, affected components and residual risk.
Controls and validation	Preventive, detective, corrective and governance controls with evidence mapping.
Action plan	Risk treatment, retest criteria and enterprise hardening recommendations.

1. Executive Summary

This threat model documents the security posture of the Enterprise DevSecOps Security Lab. The project intentionally begins with a vulnerable Flask application and then applies modern security controls across source code, CI/CD, container image build, registry storage, Kubernetes deployment, GitOps delivery, runtime detection, and audit reporting.

The model identifies major trust boundaries, high-value assets, abuse cases, potential attacker objectives, and control coverage. The analysis uses STRIDE categories to ensure broad threat coverage: spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege.

The highest-risk areas are supply-chain tampering, overprivileged Kubernetes access, injection flaws in the application, and unauthorized configuration drift. The strongest mitigations are security gates, Cosign image signing, Syft SBOM generation, ArgoCD self-healing, Kubernetes hardening, OPA Gatekeeper admission policy, Falco runtime detection, and Prometheus/Grafana observability.

Assessment conclusion

Area	Assessment result	Notes
Application security	Partially remediated / intentionally vulnerable for lab evidence	SQL injection, command injection, and unsafe rendering are documented with remediation examples and retest criteria.
CI/CD security	Strong for private lab	SAST, SCA, Trivy, security gates, SARIF artifacts, and Scorecard workflow are included.
Container security	Strong baseline	Non-root image, healthcheck, Trivy image scan, Cosign signing, SBOM and provenance evidence.
Kubernetes security	Strong baseline with residual lab limitations	RBAC least privilege, restricted pod security, network policy validation, Gatekeeper and kube-bench documentation.
Runtime detection	Strong	Falco, Falcosidekick, Prometheus, Grafana dashboards and incident response runbook are documented.
Governance and auditability	Strong	Evidence index, compliance mapping, risk register, audit summary, post-mortem and retest proof exist.

2. System Overview

The system models a real enterprise secure delivery flow. A developer pushes application and infrastructure code into GitHub. GitHub Actions performs security testing and builds a Docker image. The image is pushed to GHCR, signed with Cosign, and accompanied by SBOM/provenance evidence generated by Syft and SLSA-style attestations. ArgoCD reconciles Kubernetes manifests into a Kind cluster. Falco observes runtime behavior, Falcosidekick forwards events, Prometheus stores metrics, and Grafana visualizes security posture.

Component	Role in system	Primary threats	Primary controls
Vulnerable Flask app	Application under test exposing /health, /user, /ping and /hello routes.	Injection, unsafe rendering, information disclosure, denial of service.	Parameterized queries, input validation, security headers, safe subprocess usage, container isolation.
GitHub repository	Source of truth for application code, Dockerfile, workflows, Kubernetes manifests and documentation.	Tampering, unauthorized commit, secret leakage, repudiation.	Branch protection, PR review, signed commits, least-privilege tokens, workflow artifacts.
GitHub Actions	CI/CD execution layer for SAST, SCA, Trivy, image build, SARIF upload and security gates.	Malicious workflow modification, dependency poisoning, credential abuse.	Pinned actions, limited permissions, security-gates workflow, scorecard review, artifact retention.
GHCR container registry	Stores built image, image digest and supply-chain artifacts.	Image substitution, unsigned image, tampering with tags.	Cosign signing, digest pinning, SBOM/provenance attestation, registry access control.
ArgoCD	GitOps controller that reconciles k8s/base manifests into the Kind cluster.	Repo spoofing, drift, privilege abuse, unauthorized sync.	Private repo credentials, self-heal, prune, RBAC, audit of sync status.
Kubernetes cluster	Runtime for hardened deployment, RBAC, NetworkPolicy, Gatekeeper and observability stack.	Privilege escalation, hostPath abuse, lateral movement, exposed services.	Pod Security restricted, RBAC least privilege, NetworkPolicy, Falco, kube-bench, Gatekeeper.
Monitoring stack	Falco, Falcosidekick, Prometheus and Grafana provide detection and metrics.	Alert loss, dashboard tampering, metric spoofing.	Prometheus rules, dashboard-as-code, evidence screenshots, IR runbooks.

Architecture assumptions

Assumption	Security implication
The lab is private and not exposed to the public internet.	Attack simulations are safe for learning, but public exposure would require immediate remediation of vulnerable routes.
Kind is used as a local Kubernetes cluster.	The model validates Kubernetes patterns but does not claim managed-cluster production equivalence.
Some vulnerable packages and routes exist intentionally.	They are documented as lab evidence and must be fixed or isolated before production-style deployment.
GitHub is the source of truth for desired state.	Repo integrity and workflow permissions become critical trust anchors.

3. Data Flow Diagram

The diagram below shows the primary data flows, trust boundaries, artifact stores, security controls, and evidence/telemetry paths. It is also exported separately as diagrams/data-flow-diagram.svg for use in the repository.

Enterprise DevSecOps Lab - Data Flow Diagram

Threat model view showing trust boundaries, security controls, observability flows, and evidence-producing data paths.

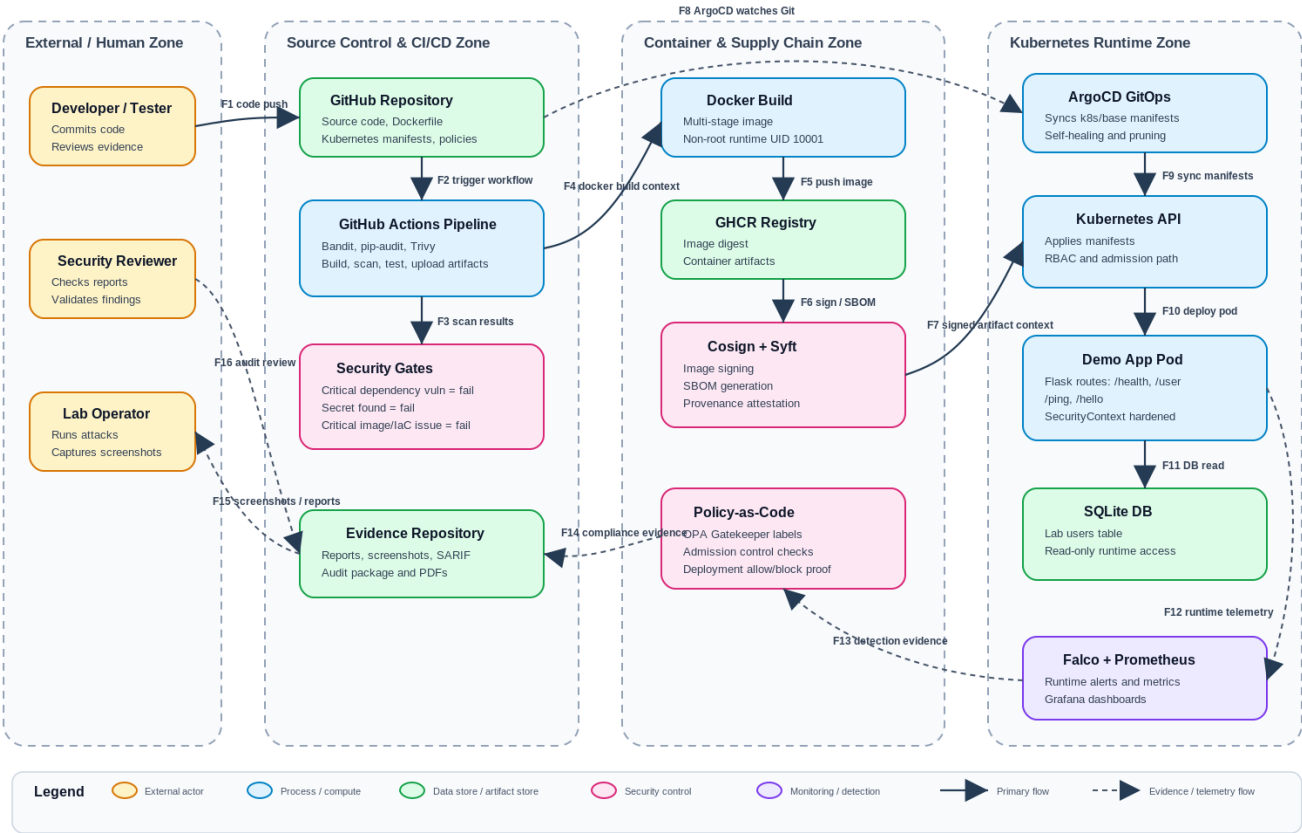


Diagram note: dashed flows represent evidence, telemetry, or audit data. Solid flows represent deployment, build, sync, or application data paths.

4. Assets and Security Objectives

Threat modeling begins by identifying assets that require protection. In this lab, assets include source code, CI/CD logic, image artifacts, Kubernetes manifests, service account permissions, SBOM/provenance evidence, and audit documentation.

Asset	Security objective	Why it matters	Protection approach
Source code	Integrity and traceability	A malicious code change can introduce backdoors or disable controls.	Version control, branch protection, reviews, SAST, artifact history.
Container image	Integrity and provenance	A replaced or tampered image can run untrusted code in Kubernetes.	Digest recording, Cosign verification, Trivy scanning, SBOM.
Kubernetes manifests	Secure desired state	Misconfigured manifests can enable privilege escalation and exposure.	IaC scanning, peer review, GitOps reconciliation, Gatekeeper policy.
Service accounts and tokens	Least privilege	Overprivileged identities can read secrets or create admin bindings.	Dedicated SAs, automount disabled, RoleBinding not ClusterRoleBinding.
SBOM and provenance	Supply-chain transparency	Security teams need to know what was built and how.	Syft SPD/CycloneDX SBOMs, attestations, signed evidence.
Security reports	Auditability	Evidence must show what was tested, what failed, and what was fixed.	Evidence index, SARIF, screenshots, retest proof, controlled storage.

Security objectives

Objective	Definition for this project	Success measure
Confidentiality	Prevent unauthorized disclosure of tokens, secrets, database content, reports and security findings.	Secrets are not committed, ServiceAccounts cannot read secrets, risky app routes are documented and remediated.
Integrity	Ensure code, manifests and container images cannot be silently altered.	Git history, ArgoCD sync, Cosign verification, SBOM/provenance and security gates are present.
Availability	Keep lab services observable and recoverable during test activity.	Resource limits, probes, monitoring dashboards and incident runbooks exist.
Accountability	Provide traceable evidence for who changed what and how controls were validated.	Evidence index, audit summary, workflow artifacts and retest proof are produced.

5. Trust Boundaries and Data Flows

A trust boundary is crossed when data or control moves between different privilege levels, ownership domains, execution contexts, or security assumptions. The most important boundaries in this project are human-to-GitHub, GitHub-to-CI, CI-to-registry, registry-to-Kubernetes, Kubernetes-to-monitoring, and runtime-to-evidence.

Flow ID	Data flow	Trust boundary crossed	STRIDE focus	Validation / evidence
F1	Developer pushes code to GitHub	Human zone to source control	Spoofing, tampering, repudiation	Commit history, branch policy, workflow run metadata.
F2-F3	GitHub Actions scans source and dependencies	Source control to CI runner	Tampering, information disclosure, DoS	Bandit report, pip-audit report, security gate summary.
F4-F6	Docker build, registry push, signing and SBOM generation	CI/CD to registry/supply-chain zone	Tampering, spoofing, information disclosure	Image digest, Trivy report, Cosign verification, Syft SBOM.
F8-F10	ArgoCD syncs Kubernetes manifests	GitHub to cluster control plane	Spoofing, tampering, elevation of privilege	ArgoCD resource tree, self-healing proof, deployment state.
F11	App pod reads SQLite database	Pod runtime to data store	Information disclosure, tampering	Read-only root filesystem proof, SQLite mode=ro, app logs.
F12-F13	Falco emits detections to Prometheus/Grafana	Runtime to monitoring	Repudiation, information disclosure, alert tampering	Falco logs, Prometheus queries, Grafana dashboard screenshots.

High-risk trust boundary observations

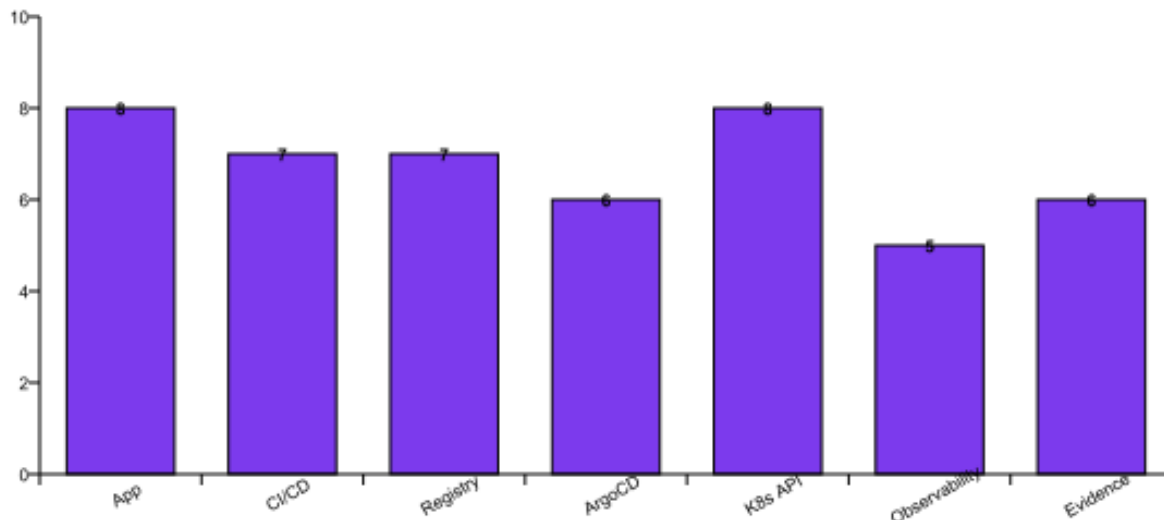
Boundary	Why it is sensitive	Threats	Required controls
GitHub to Actions runner	Workflow executes code with repository context and access to tokens.	Tampering, spoofing, secret leakage.	Minimal permissions, pinned actions, artifact review, Scorecard checks.
CI runner to GHCR	Built image becomes deployable artifact.	Image substitution, tag tampering.	Digest validation, Cosign signing, registry access control.
GitHub to ArgoCD	Git state drives Kubernetes desired state.	Repo spoofing, malicious manifest update.	Private repo secret, branch protection, ArgoCD RBAC, self-heal.
Kubernetes API to pods	Admission path decides what workloads can run.	Privilege escalation, hostPath abuse.	Pod Security restricted, Gatekeeper, Trivy config scan.
Runtime to observability	Detections become operational evidence.	Alert loss or dashboard tampering.	Falco rules, Prometheus rules, dashboard-as-code, IR runbook.

6. Attack Surface Analysis

The attack surface includes application routes, CI/CD workflow files, third-party actions, dependencies, Dockerfile, image registry tags, Kubernetes API, ArgoCD configuration, service accounts, network paths, admission controllers, and observability interfaces.

Threat exposure by component

Qualitative score reflects attack surface, privilege, trust boundary crossings, and detection coverage.



Surface	Potential attack	Risk driver	Control evidence
Flask /user route	SQL injection returns unauthorized records.	String interpolation in SQL query.	SQL injection finding, secure code remediation examples, AppSec tests.
Flask /ping route	Command injection executes shell commands.	shell=True and unsanitized host input.	Command injection finding, Bandit summary, test_routes.py.
Flask /hello route	Unsafe template rendering evaluates user-controlled input.	render_template_string with formatted input.	Template injection finding, secure rendering examples.
requirements.txt	Outdated vulnerable dependencies become exploitable.	Pinned old packages for lab SCA evidence.	pip-audit summary, dependency vulnerability finding, SCA gate.
Dockerfile	Image misconfig enables root or writable filesystem.	Container runtime settings can weaken isolation.	Docker hardening, Trivy image/config scan, non-root proof.
Kubernetes RBAC	ServiceAccount receives cluster-admin.	ClusterRoleBinding grants excessive permissions.	RBAC review, can-i validation, least privilege fix.
Privileged pod path	hostPath + privileged enables container escape pattern.	Pod can see node filesystem.	Pod security validation, Falco detection, restricted namespace.
NetworkPolicy	Untrusted pods reach app/service laterally.	Flat pod network without default deny.	Calico validation, allowed/blocked curl pod proof.

7. Threat Register

The following register summarizes key threats discovered during modeling. Risk is qualitative and based on exploitability in the private lab, potential enterprise impact, and available controls.

ID	STRIDE	Threat scenario	Affected component	Impact	Controls / mitigations	Residual risk
TM-01	Spoofing	Attacker impersonates trusted developer or workflow actor to push malicious code or alter CI/CD logic.	GitHub repo, GitHub Actions	Unauthorized code execution and supply-chain compromise.	Branch protection, PR approvals, least-privilege workflow token, Scorecard checks.	Medium - private lab does not enforce enterprise IdP/MFA evidence.
TM-02	Tampering	Container image tag is replaced after scan, causing Kubernetes to deploy an unverified image.	GHCR, ArgoCD, Kubernetes	Runtime executes code different from reviewed artifact.	Image digest tracking, Cosign verification, SBOM/provenance attestations.	Medium - digest pinning should be enforced before production.
TM-03	Repudiation	A manual cluster change is performed without a clear audit trail.	Kubernetes, ArgoCD	Weak accountability and difficulty proving change history.	ArgoCD self-heal, Git as source of truth, workflow artifacts and evidence index.	Low - Kind lab lacks centralized audit log retention.
TM-04	Information Disclosure	Injection flaw or unsafe rendering exposes application internals, users table, or runtime data.	Flask app, SQLite DB	Unauthorized data exposure and attack path discovery.	Parameterized SQL, safe rendering, security headers, SAST tests.	Medium - vulnerable route remains intentionally documented for lab evidence.
TM-05	Denial of Service	High request volume or resource-intensive input exhausts app pod CPU/memory.	Flask app, Kubernetes deployment	Service unavailable, pod restarts, degraded testing environment.	Resource requests/limits, liveness/readiness probes, monitoring.	Medium - no full rate limiter/load balancer in private lab.
TM-06	Elevation of Privilege	Compromised pod or overprivileged service account creates privileged workload or reads secrets.	Kubernetes RBAC, Pod Security	Cluster compromise and credential exposure.	RBAC least privilege, automount disabled, Pod Security restricted, Gatekeeper, Falco.	Low to Medium - depends on ongoing policy enforcement.
TM-07	Tampering	Kubernetes manifest is modified to remove hardening controls such as readOnlyRootFilesystem or seccomp.	k8s/base manifests	Weakens runtime isolation and violates hardening baseline.	Trivy config scan, PR review, GitOps, Gatekeeper policies.	Medium - not every control is enforced by admission policy yet.
TM-08	Information Disclosure	Secret accidentally committed or exposed through workflow logs.	GitHub repo, Actions logs	Credential leakage and registry/repo compromise.	Trivy secret scan, least-privilege token permissions, artifact review.	Medium - historical secret scanning and enterprise DLP not simulated.
TM-09	Repudiation	Security findings are fixed without documented retest proof.	Evidence and audit package	Inability to prove remediation closure.	Retest proof PDF, evidence index, control validation matrix.	Low - depends on documentation discipline.

8. Control Coverage

Threat controls were grouped into preventive, detective, corrective and governance layers. The goal is not only to block issues, but also to produce repeatable evidence for audit and remediation verification.

Control layer	Implemented controls	Threats reduced	Evidence
Preventive	Security gates, least-privilege RBAC, Pod Security restricted, NetworkPolicy default deny, Gatekeeper admission policy, non-root container.	Tampering, spoofing, information disclosure, elevation of privilege.	security-gates.yml, RBAC review, pod security validation, network policy validation.
Detective	Bandit, pip-audit, Trivy, Falco, Falcosidekick, Prometheus rules, Grafana dashboards.	Injection, vulnerable dependencies, runtime attack behavior, misconfigurations.	SARIF artifacts, Falco logs, Prometheus query result, dashboard screenshots.
Corrective	Secure code remediation examples, retest proof, ArgoCD self-heal, vulnerability exceptions and VEX notes.	Residual risk, configuration drift, unfixed exceptions.	Retest report, VEX notes, exception register, ArgoCD self-heal screenshots.
Governance	Evidence index, compliance mapping, audit summary, risk register, post-mortem.	Repudiation and audit failure.	PDF package, source markdown/CSV files, screenshots and reports.

Control strengths

Strength	Explanation
Defense in depth	The project does not rely on one tool. It layers code scanning, dependency scanning, image scanning, policy-as-code, runtime detection and monitoring.
Shift-left and runtime coverage	SAST/SCA/Trivy catch issues before deployment while Falco/Prometheus/Grafana provide runtime detection.
Evidence-driven validation	Each major control has evidence screenshots, reports, retest criteria and a documented owner.
Enterprise audit readiness	The project includes compliance mapping, control validation, risk register, post-mortem and exception/VEX documentation.

Control maturity notes

Maturity area	Current lab maturity	Enterprise next step
Security gates	Implemented with clear failure thresholds for critical findings.	Make gates mandatory status checks on protected branches.
Supply chain evidence	Cosign, Syft, SBOM and provenance are documented.	Use trusted builder provenance and admission-time signature verification.
Kubernetes enforcement	Pod Security, NetworkPolicy, RBAC and Gatekeeper are validated.	Expand policies to enforce image digest and signature requirements.
Detection response	Falco detections and IR runbooks exist.	Route alerts to a SIEM/ticketing workflow with ownership SLAs.

9. Risk Treatment Plan

Risk area	Treatment decision	Reason	Next action
Intentionally vulnerable Flask routes	Accept temporarily for lab demonstration	Vulnerabilities are required for detection, documentation and remediation evidence.	Maintain warnings in README and never expose to public internet.
Legacy dependencies	Mitigate and track	Old packages demonstrate SCA findings; production builds should update versions.	Create fixed requirements file and compare pip-audit results after remediation.
Image tag deployment	Mitigate	Tags can move and do not guarantee artifact immutability.	Use digest-pinned images in production-style manifests.
Local SLSA-style provenance	Accept with documented limitation	Local lab provenance demonstrates concept but is not equivalent to trusted builder provenance.	Generate provenance in GitHub Actions for stronger SLSA alignment.
Kind cluster limitations	Accept for private lab	Kind does not represent a managed enterprise Kubernetes control plane.	Document limitations and avoid overclaiming production compliance.

Residual risk statement

The project provides strong evidence for secure engineering practices in a private lab. Remaining risks are mostly lab-scope limitations: lack of enterprise identity provider integration, limited centralized audit logging, local provenance generation, and intentional vulnerable routes used for demonstration. These risks are acceptable for the stated purpose but should be remediated or replaced before production use.

10. Validation Checklist

Validation area	Expected proof	Status
Application security	SQL injection, command injection, unsafe rendering findings and remediation tests.	Documented
CI/CD gates	security-gates.yml blocks critical dependency, secret, container and Kubernetes issues.	Implemented
Supply chain	Cosign signature verification, SBOM package count, provenance summary.	Implemented
Kubernetes hardening	SecurityContext, RBAC, Pod Security, NetworkPolicy and kube-bench result documentation.	Implemented
Detection engineering	Falco alerts, Prometheus rules, Grafana dashboard, IR runbook.	Implemented
Compliance and audit	Compliance mapping, control validation matrix, evidence index, risk register.	Implemented

11. Recommendations

Priority	Recommendation	Enterprise rationale
P1	Pin images by digest in Kubernetes manifests.	Prevents tag substitution and improves reproducibility.
P1	Move SLSA provenance generation into GitHub Actions.	Provides stronger builder identity and tamper-resistant provenance.
P1	Convert vulnerable routes into fixed routes and keep vulnerable examples in a separate lab branch.	Allows production-like branch to pass gates while retaining learning evidence.
P2	Add Kyverno or Sigstore Policy Controller for signature enforcement.	Gatekeeper label checks prove policy-as-code, but signature verification should be enforced at admission.
P2	Add audit log forwarding to SIEM-style storage.	Improves repudiation controls and incident reconstruction.
P2	Add rate limiting and request body limits.	Improves DoS resilience for application layer.